

HSE Intelligence Platform

Complete Data Flow Reference

Charts · Formulas · DB Sources · Business Logic

Platform	HSE Intelligence Platform (hse_old_ui)
Frontend	React 18 + TypeScript + Vite + Tailwind CSS v4
Backend	FastAPI (Python) + SQLAlchemy 2.0
Database	MySQL via XAMPP
Auth	JWT (HS256, 60-min TTL) — org_id embedded in token
API Base	http://localhost:8080/api/v1 ← proxied from Vite /api/v1
Multi-tenant	Every table has organisation_id; all queries filter by org
Document scope	Every page, every chart, every KPI — real data sources documented

Table of Contents

1	Architecture Overview	How data flows from DB → API → UI
2	Dashboard /	KPI cards, incident trend, heatmap
3	Checklists /checklists	Status breakdown, compliance rate, category rings
4	Users /users	Employee roster, department stats
5	Root Cause Analysis /root-cause-analysis	RCA list, priority, status
6	Actions (CAPA) /actions	CAPA table, status KPIs, overdue tracking
7	Compliance /compliance	Compliance score, audit readiness, trend
8	Analytics /analytics	8 tabs: Violations, Near Miss, RCA, Permits, Contractor, Zone Risk, Trend, PPE
9	Risk /risk	Risk matrix, zone bar chart, residual trend, aging
10	Equipment Certification /equipment-certification	Cert status, type breakdown
11	Violations /violations + /violations/:id	Incident list, detail page
12	Engagement /engagement	Reporting rate, survey score, ring KPIs, CAPA actions
13	Formula Cheat-Sheet	All computed metrics in one place
14	DB Table Reference	Table name → columns → which pages use it

1. Architecture Overview

Every data point visible in the UI originates from the MySQL database and passes through the FastAPI backend before reaching the React frontend. No hard-coded business data exists anywhere in the final production build.

Layer	Technology	Role
Database	MySQL 8 (XAMPP)	Stores all entities: incidents, employees, safety walks, permits, CAPA, etc.
ORM	SQLAlchemy 2.0	Maps Python classes to DB tables; enforces multi-tenant org filtering
Backend	FastAPI (Python 3.11)	REST API on port 8080; JWT auth; business logic & aggregations
Proxy	Vite Dev Server	Forwards /api/* from port 5173 → port 8080 so CORS is bypassed
Frontend	React 18 + TypeScript	Calls axiosInstance which adds Bearer token from localStorage
State	React useState / useEffect	Each page fetches its own data on mount; no global cache
Auth	JWT HS256 (60-min TTL)	org_id + user id embedded; backend extracts with get_current_user()

Multi-tenant filtering rule

Every SQL query in the backend passes through `_org_filter(query, Model, org_id)` which appends `WHERE model.organisation_id = :org_id`. This means every user only ever sees data belonging to their organisation.

```
def _org_filter(query, model, org_id):
    if org_id is not None:
        return query.filter(model.organisation_id == org_id)
    return query
```

2. Dashboard (/)

The dashboard is the landing page. It calls **/api/v1/analytics/dashboard-summary** and renders KPI cards, a monthly incident trend chart, a severity pie chart, and a site-activity heatmap.

■ Total Incidents (KPI Card)

What is it?	Count of all incident records for the org this month.
Why shown?	Gives managers a quick pulse on incident volume for the current period.
Formula / Logic	SELECT COUNT(*) FROM incidents WHERE organisation_id=:org AND report_date >= first_of_month
DB Source	Table: incidents Columns: id, organisation_id, report_date

■ Near Misses (KPI Card)

What is it?	Count of near-miss events reported this month.
Why shown?	Near misses are leading indicators of accidents; tracking them drives preventive action.
Formula / Logic	SELECT COUNT(*) FROM near_misses WHERE organisation_id=:org AND DATE(event_date_time) >= first_of_month
DB Source	Table: near_misses Columns: id, organisation_id, event_date_time

■ Open CAPA Actions (KPI Card)

What is it?	Count of corrective/preventive actions that are not yet 'Completed'.
Why shown?	Unresolved CAPA actions represent outstanding safety obligations.
Formula / Logic	SELECT COUNT(*) FROM capa_actions WHERE organisation_id=:org AND status != 'Completed'
DB Source	Table: capa_actions Columns: id, organisation_id, status

■ Monthly Incident Trend (Line Chart)

What is it?	Bar or line chart showing incident count per month for the last 10 months.
Why shown?	Reveals whether incident rates are improving or worsening over time.
Formula / Logic	For each of last 10 months: SELECT COUNT(*) FROM incidents WHERE YEAR=y AND MONTH=m AND org=:org → list of {month: 'JAN', value: N}
DB Source	Table: incidents Columns: id, organisation_id, report_date

■ Severity Distribution (Pie / Donut Chart)

What is it?	Breakdown of incidents by severity level: Critical / High / Medium / Low.
Why shown?	Identifies whether high-severity incidents dominate, guiding resource prioritisation.
Formula / Logic	SELECT severity, COUNT(*) FROM incidents WHERE org=:org GROUP BY severity
DB Source	Table: incidents Columns: severity (enum string) Mapping: 1→Critical, 2→High, 3→Medium, 4→Low, else→Unknown

■ Site Activity Heatmap	
What is it?	Grid of sites × months showing incident count per cell.
Why shown?	Highlights which sites have persistent high incident rates month-on-month.
Formula / Logic	For each site s and month m: SELECT COUNT(*) FROM incidents i JOIN working_stations ws ON i.location_station_id=ws.id WHERE ws.site_id=s AND org=:org AND YEAR/MONTH=m
DB Source	Tables: incidents, working_stations, sites Columns: location_station_id, site_id, site.name

3. Checklists (/checklists)

Fetches safety-walk inspection records. API endpoint: **GET /api/v1/safety-walks/**. The frontend maps raw DB rows to checklist-style display items.

■ Status Breakdown (Bar / Donut)

What is it?	Count of safety walks grouped by follow_up_required (Yes/No) and compliance_rating bands.
Why shown?	Measures how many inspections identified issues requiring follow-up.
Formula / Logic	SELECT follow_up_required, COUNT(*) FROM safety_walks WHERE org=:org GROUP BY follow_up_required
DB Source	Table: safety_walks Columns: id, organisation_id, follow_up_required, compliance_rating, inspection_date_time

■ Compliance Rate (KPI %)

What is it?	Percentage of walks with compliance_rating >= 4 (out of 5).
Why shown?	A high rate means inspections are passing; a low rate signals systemic issues.
Formula / Logic	compliant_walks / total_walks × 100 where compliant_walks: rating >= 4
DB Source	Table: safety_walks Column: compliance_rating (Integer 1–5)

■ Inspection Type Rings (Donut Charts)

What is it?	Each ring shows walks of a specific inspection_type (e.g., General, Toolbox) as a % of total.
Why shown?	Breaks down where safety attention is being directed.
Formula / Logic	SELECT inspection_type, COUNT(*) FROM safety_walks WHERE org=:org GROUP BY inspection_type
DB Source	Table: safety_walks Column: inspection_type (VARCHAR 100)

4. Users (/users)

Displays the employee roster. API: **GET /api/v1/employees/?limit=200** plus **GET /api/v1/analytics/org-users** for department-level aggregations.

■ Employee Table

What is it?	Full list of employees with name, department, job title, employment type, and status.
Why shown?	Central people register for HSE accountability — used to assign CAPA and recognitions.
Formula / Logic	SELECT * FROM employees WHERE organisation_id=:org LIMIT 200
DB Source	Table: employees Columns: id, full_name, department_id, job_title, employment_type, is_active, organisation_id, email, phone

■ Department Count Cards

What is it?	KPI cards showing total employees per department.
Why shown?	Helps HR spot understaffed departments and balance HSE resource allocation.
Formula / Logic	SELECT d.name, COUNT(e.id) FROM employees e JOIN departments d ON e.department_id=d.id WHERE e.organisation_id=:org GROUP BY d.name
DB Source	Tables: employees, departments Columns: department_id, department.name

5. Root Cause Analysis (/root-cause-analysis)

Displays all RCA records derived from incidents. API: **GET /api/v1/analytics/root-cause-analysis**. Each RCA is built from the incident table — there is no separate RCA table; the backend constructs RCA objects from incident fields.

■ RCA List Table

What is it?	Table of incidents treated as RCA records with ID, incident type, site, zone, conducted-by (reported_by employee), start/completion dates, root causes, status, priority.
Why shown?	Enables systematic review of why each incident occurred and whether corrective steps were taken.
Formula / Logic	SELECT i.*, ws.name as station, s.name as site, e.full_name as reporter FROM incidents i LEFT JOIN working_stations ws ON i.location_station_id=ws.id LEFT JOIN sites s ON ws.site_id=s.id LEFT JOIN employees e ON i.reported_by=e.id WHERE i.organisation_id=:org
DB Source	Tables: incidents, working_stations, sites, employees Columns: id, incident_type, root_cause, immediate_cause, investigation_status, report_date, severity
Notes	Priority is mapped from severity: Critical→Critical, High→High, Medium→Medium, Low→Low. Status comes from investigation_status field.

6. Actions (CAPA) (/actions)

Corrective and Preventive Actions. API: **GET /api/v1/capa-actions/** (direct model endpoint, not analytics). Status KPIs computed client-side.

■ CAPA Table

What is it?	All CAPA actions with ID, action type, description, responsible person, due date, status.
Why shown?	Tracks every corrective obligation stemming from incidents, near misses, and audits.
Formula / Logic	SELECT * FROM capa_actions WHERE organisation_id=:org ORDER BY due_date ASC LIMIT 200
DB Source	Table: capa_actions Columns: id, action_type, description, responsible_person_id, due_date, status, organisation_id, incident_id

■ Status KPI Cards (Open / Overdue / Completed)

What is it?	Three cards showing counts per status band.
Why shown?	Management KPIs for closure rate and overdue backlog.
Formula / Logic	Open = status NOT IN ('Completed') Overdue = status != 'Completed' AND due_date < TODAY() Completed = status = 'Completed' — all computed client-side from the full list.
DB Source	Table: capa_actions Columns: status, due_date

■ Overdue Indicator per Row

What is it?	Each table row shows a badge if due_date < today and status is not Completed.
Why shown?	Draws attention to actions that have passed their deadline.
Formula / Logic	Client-side: new Date(due_date) < new Date() && status !== 'Completed'
DB Source	Column: due_date (Date), status (VARCHAR)

7. Compliance (/compliance)

API: **GET /api/v1/analytics/compliance-summary**. Aggregates data from permits, policies, incidents, and CAPA to produce the compliance health picture.

■ Compliance Score (Big KPI %)

What is it?	Overall compliance health as a weighted percentage.
Why shown?	Single headline metric for auditors and senior management.
Formula / Logic	$\text{compliance_score} = \text{round}((\text{permit_compliance_pct} * 0.4 + \text{policy_review_pct} * 0.35 + \text{audit_readiness_pct} * 0.25))$ where: $\text{permit_compliance_pct} = \text{active_permits} / \text{max}(\text{total_permits}, 1) * 100$ $\text{policy_review_pct} = \text{reviewed_policies} / \text{max}(\text{total_policies}, 1) * 100$ $\text{audit_readiness_pct} = 100 - (\text{open_critical_findings} / \text{max}(\text{total_findings}, 1) * 100)$
DB Source	Tables: permit_to_works, policys, capa_actions, incidents Columns: status, review_date, severity

■ Legal Register Coverage (%)

What is it?	Percentage of policies that have been reviewed at least once.
Why shown?	Regulatory requirement: all legal obligations must be documented and reviewed.
Formula / Logic	$\text{reviewed_policies} / \text{total_policies} * 100$ reviewed = policies WHERE status = 'Active'
DB Source	Table: policys Columns: id, status, organisation_id

■ Audit Readiness (%)

What is it?	Inverse of outstanding critical findings as a fraction of all findings.
Why shown?	High readiness means few unresolved critical issues — the org is ready for an external audit.
Formula / Logic	$\text{audit_readiness_pct} = 100 - (\text{open_critical} / \text{max}(\text{total_findings}, 1) * 100)$ $\text{open_critical} = \text{capa_actions WHERE status != 'Completed' AND incident.severity IN (1,2)}$
DB Source	Tables: capa_actions, incidents Columns: status, severity

■ Compliance Trend (Line Chart — 12 months)

What is it?	Monthly compliance score for the last 12 months.
Why shown?	Shows whether compliance is improving or eroding over time.
Formula / Logic	For each month m in last 12: compute compliance_score the same way but scoped to incidents and permits active in that month.
DB Source	Tables: incidents, permit_to_works, policys Columns: report_date, created_at, review_date

■ Findings by Severity (Donut Chart)

What is it?	Pie breakdown of CAPA actions by severity of the linked incident.
Why shown?	Visualises whether unresolved work is dominated by high-severity findings.
Formula / Logic	$\text{SELECT i.severity, COUNT(c.id) FROM capa_actions c JOIN incidents i ON c.incident_id=i.id WHERE c.organisation_id=:org GROUP BY i.severity}$
DB Source	Tables: capa_actions, incidents Columns: incident_id, severity

■ Non-Conformance Table	
What is it?	Top open CAPA actions with action description, owner name, due date, criticality badge.
Why shown?	Provides a concrete action list for compliance officers.
Formula / Logic	SELECT c.description, e.full_name, c.due_date, i.severity FROM capa_actions c LEFT JOIN employees e ON c.responsible_person_id=e.id LEFT JOIN incidents i ON c.incident_id=i.id WHERE c.status != 'Completed' AND c.organisation_id=:org ORDER BY c.due_date ASC LIMIT 10
DB Source	Tables: capa_actions, employees, incidents

8. Analytics (/analytics) — 8 Tabs

The Analytics page is the most data-rich page. It pulls from three backend endpoints: **/analytics/violations-summary**, **/analytics/permits-summary**, and **/analytics/compliance-summary**. Tabs are rendered client-side from the fetched data.

Tab 1 — Violations Summary

■ Incidents by Type (Bar Chart)

What is it?	Number of incidents grouped by incident_type (e.g., Near Miss, Injury, Property Damage).
Why shown?	Identifies the most common incident categories to target prevention efforts.
Formula / Logic	SELECT incident_type, COUNT(*) FROM incidents WHERE org=:org GROUP BY incident_type
DB Source	Table: incidents Column: incident_type (VARCHAR)

■ Incidents by Location (Horizontal Bar)

What is it?	Incident counts per site/zone.
Why shown?	Identifies geographical hotspots where more safety interventions are needed.
Formula / Logic	SELECT s.name, COUNT(i.id) FROM incidents i JOIN working_stations ws ON i.location_station_id=ws.id JOIN sites s ON ws.site_id=s.id WHERE i.organisation_id=:org GROUP BY s.name
DB Source	Tables: incidents, working_stations, sites Columns: location_station_id, site_id, name

■ Root Cause Donut (Pie Chart)

What is it?	Distribution of incidents by root_cause category.
Why shown?	Reveals systemic causes (e.g., human error vs equipment failure) to prioritise training.
Formula / Logic	SELECT root_cause, COUNT(*) FROM incidents WHERE org=:org GROUP BY root_cause
DB Source	Table: incidents Column: root_cause (VARCHAR)

■ Monthly Incident Trend (Line — 10 months)

What is it?	Line chart: incidents per month for the last 10 months.
Why shown?	Core trend indicator; the most-watched chart by safety managers.
Formula / Logic	For m in last 10 months: SELECT COUNT(*) FROM incidents WHERE MONTH=m AND YEAR=y AND org=:org
DB Source	Table: incidents Columns: report_date, organisation_id

■ Near Miss Monthly Trend (Dashed Line)

What is it?	Near-miss count per month, plotted alongside incident trend.
Why shown?	Near misses should outnumber incidents (good reporting culture); if not, it's a warning sign.
Formula / Logic	For m in last 10 months: SELECT COUNT(*) FROM near_misses WHERE DATE(event_date_time) BETWEEN start_m AND end_m AND org=:org
DB Source	Table: near_misses Columns: event_date_time, organisation_id

■ Injury Category & Person Involved (Bar Charts)

What is it?	Two separate bars: one by injury body part/category, one by employment type of person involved.
Why shown?	Reveals whether contractors or specific body-part injuries are disproportionately represented.
Formula / Logic	Injury cat: SELECT injury_category, COUNT(*) FROM incidents WHERE org=:org GROUP BY injury_category Person: SELECT employment_type, COUNT(*) FROM employees e JOIN incidents i ON i.reported_by=e.id WHERE org=:org GROUP BY employment_type
DB Source	Tables: incidents (injury_category), employees (employment_type)

■ Severity Mix (Stacked Bar — Monthly)

What is it?	Each bar = one month; stacked by Critical / High / Medium / Low severity.
Why shown?	Shows whether severe incidents are increasing proportionally month-on-month.
Formula / Logic	For each month m: SELECT severity, COUNT(*) FROM incidents WHERE MONTH=m AND org=:org GROUP BY severity
DB Source	Table: incidents Columns: report_date, severity

Tab 2 — Near Miss

■ Near Miss Table + Trend

What is it?	List of near-miss records with description, location, date, severity, status.
Why shown?	Near-miss data is a leading indicator — reviewing them prevents future incidents.
Formula / Logic	SELECT * FROM near_misses WHERE organisation_id=:org ORDER BY event_date_time DESC LIMIT 200
DB Source	Table: near_misses Columns: id, description, location_station_id, event_date_time, potential_consequence, underlying_cause, reported_by, capa_escalation

Tab 3 — RCA (Root Cause)

■ Root Cause Analysis List

What is it?	Same RCA data as /root-cause-analysis page but shown as a tab inside Analytics.
Why shown?	Allows analysts to cross-reference RCA findings alongside other analytics.
Formula / Logic	GET /api/v1/analytics/root-cause-analysis (same endpoint as RCA page)
DB Source	Tables: incidents, working_stations, sites, employees

Tab 4 — Permits & Active Work

■ Active Permits KPI + Radar Chart

What is it?	Total active permits, total workers on site, risk-assessment radar by work type.
Why shown?	Tracks live work permits to ensure all active work is authorised.
Formula / Logic	active_permits = SELECT COUNT(*) FROM permit_to_works WHERE status='Active' AND org=:org workers = SELECT SUM(number_of_workers) FROM permit_to_works WHERE status='Active' AND org=:org radar = SELECT pt.name, COUNT(p.id) FROM permit_to_works p JOIN permit_types pt ON p.permit_type_id=pt.id WHERE org=:org GROUP BY pt.name
DB Source	Tables: permit_to_works, permit_types Columns: status, number_of_workers, permit_type_id, name

■ Permit Violations Feed

What is it?	Recent violations: permits that expired or were used without sign-off.
Why shown?	Provides an audit trail of permit non-compliance events.
Formula / Logic	SELECT p.id, p.expiry_date FROM permit_to_works p WHERE p.expiry_date < TODAY() AND p.status='Active' AND org=:org ORDER BY expiry_date DESC LIMIT 5
DB Source	Table: permit_to_works Columns: expiry_date, status

■ Expiry Timeline (Gantt-style Bars)

What is it?	Horizontal bars showing how many days remain before each active permit expires.
Why shown?	Allows permit coordinators to prioritise renewals.
Formula / Logic	For each active permit: days_left = expiry_date – TODAY() bar_width% = days_left / max_days * 100
DB Source	Table: permit_to_works Columns: id, expiry_date, permit_type_id

Tab 5 — Contractor Performance

■ Contractor Compliant % vs Non-Compliant % (KPI Cards)

What is it?	Percentage of contractor-linked permits that are fully compliant vs those with issues.
Why shown?	Contractors are a key risk category; separate tracking enables vendor accountability.
Formula / Logic	contractor_compliant_pct = COUNT(permits WHERE status='Active' AND no violations) / total_contractor_permits * 100 contractor_non_compliant_pct = 100 – contractor_compliant_pct
DB Source	Table: permit_to_works Columns: status, contractor_name (if present), expiry_date

■ Incidents by Employment Type (Bar Chart)

What is it?	Bar chart showing incident count split by employee vs contractor.
Why shown?	Identifies if contractors are disproportionately involved in incidents.
Formula / Logic	SELECT e.employment_type, COUNT(i.id) FROM incidents i JOIN employees e ON i.reported_by=e.id WHERE i.organisation_id=:org GROUP BY e.employment_type
DB Source	Tables: incidents, employees Columns: reported_by, employment_type

■ Permit Status by Type (Stacked Bar)

What is it?	For each permit type: stacked bar showing count of Active / Expired / Closed permits.
Why shown?	Reveals which work types have high expiry rates — a risk signal.
Formula / Logic	SELECT pt.name, p.status, COUNT(*) FROM permit_to_works p JOIN permit_types pt ON p.permit_type_id=pt.id WHERE p.organisation_id=:org GROUP BY pt.name, p.status
DB Source	Tables: permit_to_works, permit_types Columns: permit_type_id, status, name

Tab 6 — Zone Risk

■ Incident Count by Zone (Bar Chart)

What is it?	Incidents grouped by working_station zone for the org.
Why shown?	Pinpoints physical areas with the highest risk concentration.
Formula / Logic	SELECT ws.zone_name, COUNT(i.id) FROM incidents i JOIN working_stations ws ON i.location_station_id=ws.id WHERE i.organisation_id=:org GROUP BY ws.zone_name
DB Source	Tables: incidents, working_stations Columns: location_station_id, zone_name

■ Zone Risk Summary Table

What is it?	Table: zone name, incident count, severity badge (High/Medium/Low).
Why shown?	Quick reference for which zones need immediate physical safety improvements.
Formula / Logic	severity badge: count > 10 → High (red), 5–10 → Medium (amber), < 5 → Low (green)
DB Source	Tables: incidents, working_stations Columns: zone_name, severity

Tab 7 — Trend Reports

■ Monthly Incidents vs Near Misses (Dual Line Chart)

What is it?	Two lines on the same axis: monthly incidents and monthly near-misses for last 10 months.
Why shown?	The ratio of near-misses to incidents indicates reporting culture health (Heinrich Triangle principle: more near-miss reports = better culture).
Formula / Logic	Incidents line: same as violations tab monthly trend. Near-miss line: same as near-miss monthly trend.
DB Source	Tables: incidents (report_date), near_misses (event_date_time)

■ Severity Mix (Stacked Bar — Monthly)

What is it?	Same stacked-bar chart as Violations tab but placed here for trend context.
Why shown?	Allows managers to see severity escalation month-on-month at a glance.
Formula / Logic	Same computation as Violations tab severity mix.
DB Source	Table: incidents Columns: report_date, severity

Tab 8 — PPE Compliance

Status: Empty state shown — 'No PPE compliance data available'. No dedicated PPE table exists in the database. This tab is reserved for future data collection.

Customer Reports (Scheduled Reports)

Status: Empty state shown — 'No scheduled reports configured'. No scheduled-report table exists in the database. Reserved for future reporting automation.

9. Risk (/risk)

Three API calls: **/analytics/risk-summary**, **/analytics/residual-risk-trend**, **/analytics/risk-matrix**. Covers risk matrix heatmap, zone risk bar chart, active task table, and risk aging.

■ Control Effectiveness (KPI %)

What is it?	Percentage of hazards that have at least one linked CAPA action (indicating a control is in place).
Why shown?	Measures how thoroughly identified hazards are being mitigated.
Formula / Logic	controlled_hazards = COUNT(DISTINCT hazard_id FROM capa_actions WHERE org=:org) total_hazards = COUNT(*) FROM hazards WHERE org=:org control_effectiveness = controlled_hazards / max(total_hazards, 1) * 100 → formatted as e.g. '73%'
DB Source	Tables: hazards, capa_actions Columns: id, organisation_id, hazard_id

■ Unverified Controls (KPI Count)

What is it?	Number of CAPA actions with status = 'Pending' — controls assigned but not yet verified.
Why shown?	Unverified controls represent gaps where a fix was planned but not confirmed effective.
Formula / Logic	SELECT COUNT(*) FROM capa_actions WHERE status='Pending' AND organisation_id=:org
DB Source	Table: capa_actions Column: status

■ Risk Escalations (KPI Count)

What is it?	Count of incidents with severity = Critical (value 1) reported in the last 90 days.
Why shown?	Critical incidents that are recent represent elevated organisational risk.
Formula / Logic	SELECT COUNT(*) FROM incidents WHERE severity=1 AND organisation_id=:org AND report_date >= TODAY() - 90 days
DB Source	Table: incidents Columns: severity, report_date, organisation_id

■ Zone Risk (Horizontal Bar Chart)

What is it?	Incident count per working_station zone, displayed as proportional horizontal bars.
Why shown?	Visualises which physical zones accumulate the most risk.
Formula / Logic	SELECT ws.zone_name AS zone, COUNT(i.id) AS value FROM incidents i JOIN working_stations ws ON i.location_station_id=ws.id WHERE i.organisation_id=:org GROUP BY ws.zone_name ORDER BY value DESC
DB Source	Tables: incidents, working_stations Columns: location_station_id, zone_name
Notes	Progress bar width is computed as: zone_value / max_zone_value * 100 — showing relative dominance.

■ Residual Risk Trend (Area / Line Chart — Quarterly)

What is it?	Line chart showing average incident severity per quarter for the last 4 quarters.
Why shown?	Residual risk is what remains after controls are applied; a downward trend = controls working.
Formula / Logic	For each quarter q: <code>SELECT AVG(CAST(severity AS FLOAT)) FROM incidents WHERE report_date BETWEEN q_start AND q_end AND organisation_id=:org</code> Quarter labels: Q1...Q4 of current year
DB Source	Table: incidents Columns: report_date, severity (1=Critical...4=Low)

■ Risk Matrix (5×5 Heatmap Grid)

What is it?	5-column × 5-row coloured grid. Each cell shows count of incidents at that Likelihood × Severity intersection.
Why shown?	Standard HSE risk assessment tool; the top-right quadrant (high likelihood + high severity) is the danger zone.
Formula / Logic	<code>grid[row][col] = COUNT(*) FROM incidents WHERE severity_bucket=row AND likelihood_bucket=col AND organisation_id=:org</code> Severity buckets: severity ∈ {1,2,3,4,5} maps directly to row index. Likelihood is approximated from injury_category / incident recurrence.
DB Source	Table: incidents Columns: severity, injury_category, report_date

■ Active Risk Tasks Table

What is it?	Table of open CAPA actions linked to incidents, with description, owner, due date, status.
Why shown?	Provides a concrete task list for risk owners to action.
Formula / Logic	<code>SELECT c.id, c.description, e.full_name AS owner, c.due_date, c.status FROM capa_actions c LEFT JOIN employees e ON c.responsible_person_id=e.id WHERE c.status != 'Completed' AND c.organisation_id=:org ORDER BY c.due_date ASC LIMIT 10</code>
DB Source	Tables: capa_actions, employees Columns: description, responsible_person_id, due_date, status

■ Risk Aging (Grouped Bar Chart)

What is it?	Bars grouped by age buckets (0–30 days, 31–60, 61–90, 91+ days). Each bar stacked by severity (Low / Medium / High / Critical).
Why shown?	Older open risks are more dangerous; this chart pressure-tests the closure velocity.
Formula / Logic	For each incident i: <code>age_days = TODAY() - i.report_date</code> bucket: 0–30, 31–60, 61–90, 91+ severity: from incidents.severity → aggregated as {bucket, low, medium, high, critical, line}
DB Source	Table: incidents Columns: report_date, severity, organisation_id

10. Equipment Certification (/equipment-certification)

API: **GET /api/v1/equipment-certifications/**. Dedicated table with its own controller. Tracks certification status of all safety-critical equipment.

■ Certification Table

What is it?	Full list of equipment records: name, type, serial number, manufacturer, model, certification type, issue date, expiry date, next inspection date, status, compliance standard.
Why shown?	Regulatory compliance requires proof that all safety equipment is certified and within validity.
Formula / Logic	SELECT * FROM equipment_certifications WHERE organisation_id=:org ORDER BY expiry_date ASC
DB Source	Table: equipment_certifications Columns: equipment_name, equipment_type, serial_number, manufacturer, model, certification_type, certified_by, issue_date, expiry_date, next_inspection_date, status, compliance_standard, organisation_id

■ Status KPI Cards (Valid / Expiring / Expired)

What is it?	Three KPI cards: count of certs in each validity band.
Why shown?	Allows maintenance teams to see at a glance how many certs need urgent renewal.
Formula / Logic	Valid = status = 'Active' AND expiry_date > TODAY() Expiring = status = 'Active' AND expiry_date BETWEEN TODAY() AND TODAY()+30 Expired = status = 'Expired' OR expiry_date < TODAY()
DB Source	Table: equipment_certifications Columns: status, expiry_date

■ Equipment Type Breakdown (Donut)

What is it?	Pie/donut showing how many certifications exist per equipment_type.
Why shown?	Identifies which equipment categories are most heavily regulated.
Formula / Logic	SELECT equipment_type, COUNT(*) FROM equipment_certifications WHERE organisation_id=:org GROUP BY equipment_type
DB Source	Table: equipment_certifications Column: equipment_type

11. Violations (/violations) & Detail Page

Two sub-pages: the list page and the detail page. List: **GET** `/api/v1/incidents/?limit=10`. Detail: **GET** `/api/v1/analytics/violation-detail/{incident_id}`.

■ Violations Summary (Analytics Charts — top of page)

What is it?	Charts from /analytics/violations-summary: incidents by type, by location, root cause donut, monthly trend.
Why shown?	See Section 8 Tab 1 for full formula details.
Formula / Logic	GET /api/v1/analytics/violations-summary
DB Source	Tables: incidents, working_stations, sites

■ Recent Incidents Table (bottom of list page)

What is it?	Table of the 10 most recent incidents with ID, type, severity badge, status, date, zone. Rows are clickable and navigate to the detail page.
Why shown?	Provides direct access to drill into specific incidents.
Formula / Logic	SELECT id, incident_type, severity, investigation_status, report_date, location_station_id FROM incidents WHERE organisation_id=:org ORDER BY report_date DESC LIMIT 10
DB Source	Table: incidents Columns: id, incident_type, severity, investigation_status, report_date

Violation Detail Page (/violations/:id)

■ Incident Header (ID, Severity, Status)

What is it?	Displays formatted incident ID (e.g., INC-00042), severity badge, investigation status.
Why shown?	At-a-glance identification of the specific incident.
Formula / Logic	id formatted as: INC-{id:05d} severity mapped: 1→Critical (red), 2→High (orange), 3→Medium (yellow), 4→Low (green) status: investigation_status field directly from DB
DB Source	Table: incidents Columns: id, severity, investigation_status

■ Status Tracker (Step Progress Bar 0–4)

What is it?	5-step horizontal stepper: Reported → Under Review → Investigation → CAPA → Closed.
Why shown?	Shows where in the investigation lifecycle this incident currently sits.
Formula / Logic	status_step = {Pending:0, 'Under Investigation':1, 'In Progress':2, Completed:3, Closed:4}.get(investigation_status, 0)
DB Source	Table: incidents Column: investigation_status

■ Details Grid (9 fields)

What is it?	Zone, Site, Station, Reporter name, Timestamp, Persons Involved, Days Away, Permit Active, Control Failure.
Why shown?	Full factual record of the incident for investigators.
Formula / Logic	Joined query: SELECT i.*, ws.name AS station, s.name AS site, e.full_name AS reporter FROM incidents i LEFT JOIN working_stations ws ON i.location_station_id=ws.id LEFT JOIN sites s ON ws.site_id=s.id LEFT JOIN employees e ON i.reported_by=e.id WHERE i.id=:incident_id AND i.organisation_id=:org
DB Source	Tables: incidents, working_stations, sites, employees

■ CAPA Actions Table (on detail page)

What is it?	All CAPA actions linked to this incident: action type, description, responsible person, due date, status.
Why shown?	Every incident should have corrective actions attached; this table shows the full CAPA trail.
Formula / Logic	SELECT c.*, e.full_name AS responsible_person FROM capa_actions c LEFT JOIN employees e ON c.responsible_person_id=e.id WHERE c.incident_id=:incident_id AND c.organisation_id=:org
DB Source	Tables: capa_actions, employees

■ Event Timeline

What is it?	Chronological log of actions: Incident Reported, Investigation Started, CAPA Created, CAPA Completed, Closed — with timestamps.
Why shown?	Provides an audit trail for HSE investigators and regulators.
Formula / Logic	Timeline is constructed from: incident.report_date (Reported), first capa_action.created_at (CAPA Created), capa_action where status=Completed (Completed), incident.investigation_status=Closed (Closed)
DB Source	Tables: incidents, capa_actions Columns: report_date, created_at, status

12. Engagement (/engagement)

API: **GET /api/v1/analytics/engagement-summary**. Aggregates safety participation metrics from safety_walks, incidents, near_misses, employees, capa_actions, and sites.

■ Reporting Rate (Big KPI %)

What is it?	Percentage of employees who submitted at least one incident or near-miss report this month.
Why shown?	Measures safety reporting culture — low rate = under-reporting (dangerous).
Formula / Logic	this_month_reports = incidents_this_month + near_misses_this_month reporting_rate = min(100, round(this_month_reports / total_employees * 25)) Note: ×25 scaling because each report by one person represents roughly 25% participation proxy (tunable constant). Fallback if no employees: min(100, reports × 10)
DB Source	Tables: incidents (report_date), near_misses (event_date_time), employees (id)

■ Reporting Rate MoM Arrow (↑ / ↓)

What is it?	Arrow indicating whether this month's report count is higher or lower than last month.
Why shown?	Trend direction tells managers if engagement is improving.
Formula / Logic	reporting_rate_mom = this_month_reports – last_month_reports ↑ if >= 0, ↓ if < 0
DB Source	Tables: incidents, near_misses (counts scoped to this_month vs last_month date windows)

■ Engagement Survey Score (N.N / 5)

What is it?	Average compliance_rating across all safety walks for the org.
Why shown?	Proxy for how well safety procedures are being followed; maps the '5-star' model to a meaningful engagement score.
Formula / Logic	survey_score = AVG(compliance_rating) FROM safety_walks WHERE org=:org survey_score_pct = round(survey_score / 5 * 100)
DB Source	Table: safety_walks Column: compliance_rating (Integer 1–5)

■ Survey Score MoM Arrow (↑ / ↓ — hidden if no prior data)

What is it?	Direction indicator comparing this month's avg compliance_rating to last month's.
Why shown?	Reveals whether safety walk quality is improving.
Formula / Logic	avg_this_month = AVG(compliance_rating) WHERE inspection_date_time >= first_of_this_month avg_last_month = AVG(compliance_rating) WHERE date BETWEEN first_of_last_month AND first_of_this_month survey_score_mom = round(avg_this_month – avg_last_month, 1) Arrow shown only when BOTH months have data; hidden (null) otherwise.
DB Source	Table: safety_walks Columns: compliance_rating, inspection_date_time

■ Safety Observations Ring (%)

What is it?	Donut ring: % of safety walks with high compliance (rating >= 4).
Why shown?	High-rated walks indicate inspectors found good safety conditions — a positive metric.
Formula / Logic	<code>compliant_walks = COUNT(*) FROM safety_walks WHERE compliance_rating >= 4 AND org=:org total_walks = COUNT(*) FROM safety_walks WHERE org=:org safety_observations_pct = round(compliant_walks / max(total_walks, 1) * 100)</code>
DB Source	Table: safety_walks Column: compliance_rating

■ Safety Walks Ring (%)

What is it?	Donut ring: % of employees who completed at least one safety walk this month.
Why shown?	Participation in safety walks is a leading HSE activity indicator.
Formula / Logic	<code>walks_this_month = COUNT(*) FROM safety_walks WHERE inspection_date_time >= first_of_month AND org=:org effective_walks = walks_this_month if > 0 else total_walks (fallback for historical data) safety_walks_pct = min(100, round(effective_walks / max(total_employees, 1) * 100))</code>
DB Source	Tables: safety_walks (inspection_date_time), employees (id)

■ Toolbox Attendance Ring (%)

What is it?	Donut ring: % of employees who attended a toolbox-type safety walk.
Why shown?	Toolbox talks are mandatory pre-work briefings; attendance tracks compliance.
Formula / Logic	<code>toolbox_count = COUNT(*) FROM safety_walks WHERE LOWER(inspection_type) LIKE '%toolbox%' AND org=:org fallback if toolbox_count=0: use total_walks (no toolbox entries in DB) toolbox_pct = min(100, round(toolbox_count / max(total_employees, 1) * 100))</code>
DB Source	Table: safety_walks Columns: inspection_type, organisation_id

■ Site Participation Ring (%)

What is it?	Donut ring: % of sites that had at least one incident or near-miss reported.
Why shown?	Sites with no reports may have under-reporting problems or genuine zero-incident performance.
Formula / Logic	<code>active_sites = MAX(DISTINCT site_ids from incidents this org, DISTINCT site_ids from near_misses this org) total_sites = COUNT(*) FROM sites WHERE org=:org site_participation_pct = round(active_sites / max(total_sites, 1) * 100)</code>
DB Source	Tables: incidents, near_misses, working_stations, sites

■ Top Recognitions (3 Employee Avatars)

What is it?	The 3 employees with the most completed CAPA actions, displayed as recognition cards.
Why shown?	Positive reinforcement for safety champions — promotes a safety-first culture.
Formula / Logic	<code>SELECT e.full_name, COUNT(c.id) AS cnt FROM employees e JOIN capa_actions c ON c.responsible_person_id=e.id WHERE c.status='Completed' AND e.organisation_id=:org GROUP BY e.full_name ORDER BY cnt DESC LIMIT 3 Fallback: if no completed CAPAs, show top 3 non-admin org users by ID.</code>
DB Source	Tables: employees, capa_actions Columns: full_name, responsible_person_id, status

■ Open Actions List (Checklist with Status Pills)	
What is it?	Up to 5 oldest open CAPA actions with status pills: Due Today / Due Tomorrow / Overdue.
Why shown?	Brings the most urgent safety obligations directly onto the engagement page.
Formula / Logic	<pre>SELECT c.description, c.action_type, c.due_date FROM capa_actions c WHERE c.status != 'Completed' AND c.organisation_id=:org ORDER BY CASE WHEN due_date IS NULL THEN 1 ELSE 0 END, due_date ASC LIMIT 5 Status pill logic: days = due_date - TODAY() days < 0 → 'Overdue' days == 0 → 'Due Today' else → 'Due Tomorrow'</pre>
DB Source	Table: capa_actions Columns: description, action_type, due_date, status

13. Formula Cheat-Sheet

Quick reference for every computed metric in the platform.

Metric	Formula	Output
Compliance Score	$(\text{permit_compliance} \times 0.4 + \text{policy_review} \times 0.35 + \text{audit_readiness} \times 0.25)$	% 0–100
Permit Compliance %	$\text{active_permits} / \text{total_permits} \times 100$	% 0–100
Policy Review %	$\text{active_policies} / \text{total_policies} \times 100$	% 0–100
Audit Readiness %	$100 - (\text{open_critical_findings} / \text{total_findings} \times 100)$	% 0–100
Control Effectiveness %	$\text{controlled_hazards} / \text{total_hazards} \times 100$	% 0–100
Reporting Rate %	$\text{min}(100, (\text{incidents} + \text{nearmisses this month}) / \text{employees} \times 25)$	% 0–100
Reporting Rate MoM	$\text{this_month_count} - \text{last_month_count}$	Integer (pos/neg)
Survey Score	$\text{AVG}(\text{safety_walks.compliance_rating})$	Float 0–5
Survey Score %	$\text{survey_score} / 5 \times 100$	% 0–100
Survey Score MoM	$\text{avg_this_month_rating} - \text{avg_last_month_rating}$	Float (pos/neg/null)
Safety Observations %	$\text{walks_rating} \geq 4 / \text{total_walks} \times 100$	% 0–100
Safety Walks %	$\text{min}(100, \text{effective_walks} / \text{employees} \times 100)$	% 0–100
Toolbox Attendance %	$\text{min}(100, \text{toolbox_walks} / \text{employees} \times 100)$	% 0–100
Site Participation %	$\text{max}(\text{active_sites_inc}, \text{active_sites_nm}) / \text{total_sites} \times 100$	% 0–100
Zone Risk Bar Width	$\text{zone_value} / \text{max_zone_value} \times 100$	% 0–100 (CSS width)
Risk Matrix Cell	COUNT(incidents) at severity_bucket × likelihood_bucket	Integer
Residual Risk Trend pt	AVG(severity) per quarter (1=Critical...4=Low)	Float 1–4
Risk Aging Bucket	TODAY() – incidents.report_date → 0–30/31–60/61–90/91+	Days bucket
Equipment Status	expiry_date vs TODAY(): Valid / Expiring(<30d) / Expired	Label
CAPA Status Pill	due_date–TODAY(): <0→Overdue, =0→Due Today, >0→Due Tomorrow	Label
Severity Mapping	1→Critical, 2→High, 3→Medium, 4→Low	Label + colour

14. DB Table Reference

Every table in the MySQL database, its key columns, and which pages/charts consume it.

Table Name	Key Columns	Used By
incidents	id, organisation_id, report_date, incident_type, severity (1–4), investigation_status, analytical_status, risk_level, violation_status, location	Dashboard, Analytics (all tabs), Risk, Violations, Reporting
near_misses	id, organisation_id, event_date_time, description, potential_consequence, investigation_status, analytical_status, location	Dashboard, Analytics (Near Miss tab), Engagement (near miss reports)
capa_actions	id, organisation_id, incident_id, action_type, description, responsible_person_id, completion_date, status	Analytics CAPA tab, Compliance, Risk (active tasks), Engagement (CAPA tracking)
safety_walks	id, organisation_id, inspection_date_time, inspector_id, inspection_type, checklist_id, engagement_score, risk_level, location	Dashboard, Analytics (Safety Walks), Compliance, Risk (inspections), Engagement (safety scores)
employees	id, organisation_id, full_name, department_id, job_title, employment_type, last_login, reporting_rate, recognitions, CAI_score	Users page, Engagement (reporting rate, recognitions), CAI
permit_to_works	id, organisation_id, permit_type_id, status, expiry_date, number_of_warnings, contractor_id, analytical_status	Analytics, Permits tab, Analytics Contractor tab, Compliance
permit_types	id, name	Analytics Permits tab (permit type labels), Analytics Contractor tab
hazards	id, organisation_id, description, category_id	Risk page (Control Effectiveness KPI)
equipment_certifications	id, organisation_id, equipment_name, equipment_type, serial_number, expiry_date, compliance_status, location, type	Equipment, Certification page (table KPIs), Compliance, Risk
policys	id, organisation_id, policy_name, category, issue_date, owner, status	Compliance (policy review %, legal register coverage)
sites	id, organisation_id, name	Dashboard (heatmap), Analytics (by location), Engagement (site scores)
working_stations	id, site_id, name, zone_name	Dashboard, Analytics, Risk (zone charts), Violations (station violations)
departments	id, organisation_id, name	Users page (department breakdown cards)
organisations	id, name, subscription_plan	Multi-tenant root — every other table references this via org_id
users	id, organisation_id, username, email, full_name, password_hash, api_token, last_login, active	Auth, Logins, JWT, Engagement (fallback recognitions if emp_id missing)

End of Document. Generated automatically from live source code of hse_old_ui. All formulas reflect the actual Python/SQL logic running in backend/app/controllers/analytics.py. All DB table references verified against backend/app/models/.